Brachistochrone Optimization

Oscar Hernandez[1], John Rhodes
oihernandez@alaska.edu , j.rhodes@alaska.edu

December 16, 2020

Abstract

Consider a particle initially at rest which moves along a path according to the sole influence of gravity. We will confirm the first part and constructively refute the second part of Galileo Galilei's early sixteenth-century conjecture, stated below.

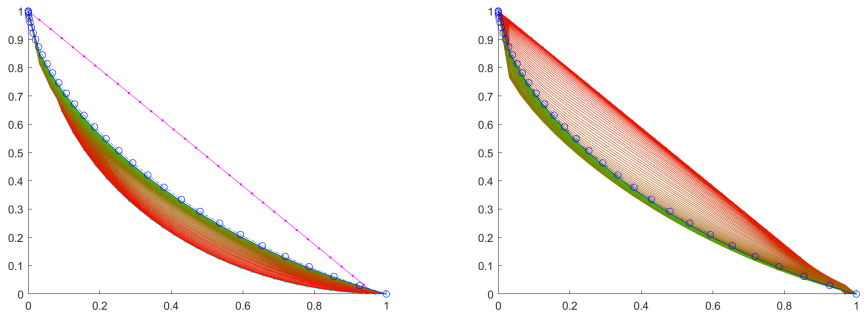
Conjecture 1. *"It is possible to infer that the quickest path of all, from one point to another, is not the shortest path, namely, a straight line, but the arc of a circle." [1]*

1 Introduction

In the late sixteenth century, Johann Bernoulli officially posed the following *brachistochrone "least-time" problem* as a 6-month challenge to "the most brilliant mathematicians in the world."

Problem 1 (Brachistochrone)**.** *"Given two points A and B in the vertical plane, what is the curve traced out by a point acted on only by gravity, which starts at A and reaches B in the shortest time?" [2]*

In Section 2, we state this problem formally. In Section 3, we derive the unique, optimal solution up to the identification of two unknown constants satisfying two equations. In Section 4, we computationally converge to it by varying any *reasonably close* guess to produce successively better numerical approximations, as shown in Figure 1 below.



(a) Red: Arc of a circle (b) Red: Straight line

Figure 1: Iteratively transform initial guesses (red) into optimal solution (green)

2 Problem

After extending the deadline for a year and half, Bernoulli published his solution along with the submissions he received from his brother Jakob Bernoulli, Gottfried Leibniz, Ehrenfried Walther von Tschirnhaus, and from an anonymous figure thought to be Sir Isaac Newton [3]. The original

[1]This article originally served as the first author's project submission to the second author's Fall 2020 course on (MATH F661) Optimization, although minor revisions have been made since then.

solutions included pure and differential geometric arguments as well as physical arguments from optics and from classical mechanics. Much of the history and problem formulation is discussed in [4].

We will highlight a modern formulation of an argument from mathematical physics. Consider a particle with mass m whose vertical displacement is given by $y : I \to M$ where $I = [\pi_x(p_0), \pi_x(p_1)]$, so that it travels along a curve $(x, y(x)) \subset M = \mathbb{R}^2$ from the starting point p_0 to the endpoint p_1. When it is clear in context, we will often write p_0 instead of its projection $\pi_x(p_0)$ onto the x-axis and similarly for p_1.

Recall the following conservation law of kinetic energy and gravitational potential energy:

Law 1.

$$\frac{1}{2}mv^2 = mgy, \quad \textit{or equivalently} \quad v = \sqrt{2gy}, \tag{1}$$

where g is the gravitational constant of acceleration. Henceforth, we shall assume the y-axis points downward.[2]

Also, recall the line element which measures infinitesimal distances in the Euclidean plane:

Definition 1.

$$ds = \sqrt{dx^2 + dy^2} = \sqrt{dx^2 \left(\left(\frac{dx}{dx} \right)^2 + \left(\frac{dy}{dx} \right)^2 \right)} = \sqrt{1^2 + y'^2}\ dx. \tag{2}$$

Then, the total time traveled is given by the integral of infinitesimal $\frac{\text{distance}}{\text{distance/time}}$ as shown below:

$$t_{01} = \int_{p_0}^{p_1} \frac{ds}{v} = \int_{p_0}^{p_1} \frac{\sqrt{1 + y'^2}}{\sqrt{2gy}}\ dx \tag{3}$$

Thus, we can state the brachistochrone problem as the following optimization problem.

Problem 2 (Brachistochrone)**.** *Find the following minimizer:*

$$\min_{y(x)}\ \frac{1}{\sqrt{2g}} \int_{p_0}^{p_1} \sqrt{\frac{1 + y'(x)^2}{y(x)}}\ dx. \tag{4}$$

This may seem tractable for an experienced student of calculus, but it was not known at the time how to take the derivative with respect to a function. Nevertheless, Leibniz and Newton independently managed to solve the problem the day they were each presented with it. In particular, Newton's solution confirmed the power of the calculus of variations which he had invented a decade earlier for the related problem of minimal resistance.

3 Analysis

Following Newton's calculus of variations approach, Leonhard Euler considered the more general problem below.

Problem 3 (Euler, 1740s)**.** *Given f, find the following minimizer $y_\star(x)$:*

$$\min_{y(x)} \int_{p_0}^{p_1} f(x, y(x), y'(x))\ dx. \tag{5}$$

[2]We obtain the same results in any other coordinate frame through a proper interpretation with vectors; in particular, we could have implemented a y-axis which points upward by reversing the sign of g and propagated its effect in subsequent uses.

Notice that this reduces to our problem when we specify the integrand $f = f_0$ as below:

$$f_0(x, y(x), y'(x)) = \sqrt{\frac{1 + y'(x)^2}{2g \cdot y(x)}}.$$

This problem differs from traditional optimization problems in that a feasible point is a curve instead of a finite-dimensional vector.

We could transform this to such a problem by searching for the finite-dimensional vector $v^n = (v_0, v_1, \ldots, v_n)$ whose i^{th} entry corresponds to something close to $v_i = y\left(x_0 + i \cdot \frac{p_1 - p_0}{n}\right)$, and study limits of v^n as n tends to $+\infty$. We revisit this approach in Section 4.

Lagrange followed Euler's work and rephrased the problem as the search for a single constant as shown below:

$$\min_{\varepsilon \in \mathbb{R}} J_\varepsilon, \quad z, y_\varepsilon \in C([p_0, p_1]) \mid z(p_0) = z(p_1) = 0, y_\varepsilon(x) = y(x) + \varepsilon \cdot z(x), J_\varepsilon = \int_{p_0}^{p_1} f(x, y_\varepsilon, y'_\varepsilon)\, dx \quad (6)$$

where the constraints on z indicate that the arguments are perturbations joining the endpoints as desired. It follows that $y(x)$ is a solution to our problem iff $\varepsilon = 0$ is a solution to this problem, an optimality condition which would become the basis for his future work on Lagrange multipliers This line of research was later refined by Weierstrass in the 1870s, by Karush in the 1930s, and popularized by Kuhn and Tucker in the 1950s.

We compute the derivative of J_ε below:

$$\begin{aligned}
\frac{dJ_\varepsilon}{d\varepsilon} &= \int_{p_0}^{p_1} \frac{d}{d\varepsilon} f(x, y_\varepsilon, y'_\varepsilon)\, dx, \text{ where } \frac{df}{d\varepsilon} = \frac{df}{dx}\frac{dx}{d\varepsilon} + \frac{df}{dy_\varepsilon}\frac{dy_\varepsilon}{d\varepsilon} + \frac{df}{dy_\varepsilon{}'}\frac{dy_\varepsilon{}'}{d\varepsilon} = \frac{df}{d\varepsilon} \cdot 0 + \frac{df}{dy_\varepsilon} \cdot z + \frac{df}{dy'_\varepsilon} \cdot z' \\
&= \int_{p_0}^{p_1} \frac{df}{dy_\varepsilon} z\, dx + \left[\frac{df}{dy'_\varepsilon} z\right]_{p_0}^{p_1} - \int_{p_0}^{p_1} z \frac{d}{dx}\frac{df}{dy'_\varepsilon} dx, \text{ by parts,}
\end{aligned}$$

and applying the optimality condition $\varepsilon = 0$ yields $f(x, y_\varepsilon, y'_\varepsilon) = f(x, y, y')$ and thus:

$$\frac{dJ_\varepsilon}{d\varepsilon} = \int_{p_0}^{p_1} z \left[\frac{df}{dy_\varepsilon} - \frac{d}{dx}\frac{df}{dy'_\varepsilon}\right] dx = 0 \text{ implies } \frac{d}{dx}\frac{df}{dy'} - \frac{df}{dy} = 0,$$

which is an Euler-Lagrange differential equation obtained by applying the fundamental lemma of calculus of variations.

Notice that $\frac{\partial f_0}{\partial x} = 0$ when we write $f_0 = (1 + y'^2)^{1/2}(2gy)^{-1/2}$ which only references x implictly. Then, we can apply Beltrami identity $f_0 - y' \frac{\partial f_0}{\partial y'} = C$ and compute $\frac{\partial f_0}{\partial y'} = y'(1 + y'^2)^{-1/2}(2gy)^{-1/2}$, and obtain the following by subtitution:

$$\begin{aligned}
f_0 \cdot \frac{\sqrt{1 + y'^2}}{\sqrt{1 + y'^2}} - y' \frac{\partial f_0}{\partial y'} &= \frac{1 - 0}{\sqrt{2gy}\sqrt{1 + y'^2}} = C \\
\left(\frac{1}{\sqrt{2gy}\sqrt{1 + y'^2}}\right)^2 \cdot y \frac{(1 + y'^2)}{C^2} - 1 &= C^2 \cdot y \frac{1 + y'^2}{C^2} - 1 \\
y\left[1 + y'^2\right] &= k^2, \qquad k = \frac{1}{2gC^2},
\end{aligned}$$

This differential equation admits the following parametric solution:

$$\begin{aligned}
x &= \frac{k^2}{2}(\theta - \sin\theta), \\
y &= \frac{k^2}{2}(1 - \cos\theta),
\end{aligned}$$

which define the cycloid, whose shape is thus optimal. However, the constant of integration C and the parameter space of angles θ are not known and are not analytically derivable. Many classical mathematicians stop here, having found the optimal shape, confident in their abilities to transform the parametric curve otherwise. As applied mathematicians, we go on to solve the two nonlinear equations for those two unknown parameters when given a choice of points p_0 and p_1 in Section 4.

4 Computation

Having expended much manual effort, we can reach a computational solution by identifying the parameters C and maximum θ present in the analytic solution.

Here, we explore a different method of computation which does not necessitate a prior analytic solution; this approach is now heavily used because of the intractability of the previous method in several dimensions.

We decompose the problem as Euler did.

$$\begin{aligned}
\min_{y(x)} \int_{p_0}^{p_1} \sqrt{\frac{1+y'^2}{2gy}}\, dx &= \min_{y(x)} \lim_{N\to\infty} \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} \sqrt{\frac{1+y'(x)^2}{2gy(x)}}\, dx && \text{by Riemann integration} \\
&\approx \min_{y(x)} \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} \sqrt{\frac{dx^2+dy^2}{2g\cdot y(x)}} && \text{for some large } N \\
&\approx \min_{y=(y_0,y_1,\ldots,y_{N-1})} \sum_{i=0}^{N-1} \sqrt{\frac{(x_{i+1}-x_i)^2+(y_{i+1}-y_i)^2}{2g\cdot \frac{y_{i+1}+y_i}{2}}} && \text{for } y_i = y(x_i),
\end{aligned}$$

where almost all of the substitutions are straight-forward; however, there are many different numerical algorithms/rules for specifying $y(x)$. We have chosen the midpoint rule.

We view this as an optimization problem with no specified constraints whose non-linear objective function is the partial sum above, and solve it through a custom implementation of the (quasi-Newton) Broyden–Fletcher–Goldfarb–Shanno algorithm described in detail below.

Code Walkthrough

We discuss some aspects of the computer program attached as Appendix A. We start by setting some hyper-parameters and "input" variables. Our main function `plot_solutions` consists of three parts:

1. calling our `gen_cycloid` method to identify the optimal cycloid's parameters
2. calling our `qNewton` method to identify the optimal vector $y_\star$
3. plotting the intermediate vector steps and the final solutions.

As discussed in Section 3, the solution to the optimality condition is a parametric curve whose geometry is defined by two parameters, the radius of the circle which traces the cycloid, and the total angle elapsed. The `gen_cycloid` methods uses MATLAB's symbolic solver; it indeed fails to derive an analytic solution but it succeeds in finding numerical estimates which experimentally agree with the optimal solution identified by the quasi-Newton method. We could have chosen to solve for the cycloid parameters with our BFGS code, but we refrained from doing so to reduce the human error by checking that different algorithms seemingly converge to the same solution.

The BFGS implementation is based on a program OH developed in a previous homework assignment for this class. The novel contribution is the numerical integration which implements the

midpoint formula. As is common, there are many ways to further "optimize" this computer program. We seek efficiency in computation, unless it interferes with the program's clarity or modularity. However, we prioritized clarity, modularity, and proof of correctness over epi-efficiency.

4.1 Generalizations

We now discuss aspects in which our code is more general than its analytic counter-part.

First, we defined the infinitesimal distance function `dist` as the Euclidean line element; we could compute the quickest path in other geometries by replacing this function with any other line element.

Second, we defined an infinitesimal numerical integral by implementing the `midpoint` formula which computes the average displacement of successive iterations k to $k+1$; we could have chosen any other formula, which each come with a set of benefits and drawbacks.

Lastly, we defined the infitesimal velocity function `vfric` according to the conservation law which accounts for a coefficient of friction c_f as follows:

Law 2.

$$v = \sqrt{2g\,(y - c_f \cdot x)}. \tag{7}$$

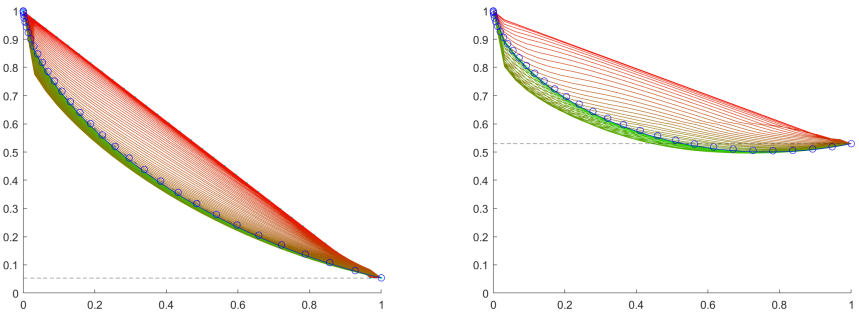
Note that we could re-define this function to account for any other force, such as drag or electromagnetic effects.

In particular, we could use this function to implement obstacles by defining the friction dynamically to be infinite over forbidden regions.

The remainder of the code has been written so as to correctly propagate these considerations.

4.2 Conclusion

We now return to the intuition which inspired this problem. It may be surprising that the shortest path between two points (namely the straight line joining them) is not the one which admits least-time travel. But it's a good guess, so we used as it as our initial guess (as shown in red). Then, we travel in the newton direction drawing each new curve in a hue which is decreasingly red and increasingly green. Sometimes the curve stays above the final height, and sometimes it drops below, depending on the choice of endpoints.



(a) (b)

Figure 2

A MATLAB Code

```
1
2  % author: oihernandez@alaska.edu
3  % Quasi-Newton solution the brachistochrone problems (1 numerical and
4  % 2 pseudo-symbolic)
5
6  clc; clf; clear;
7  format long;
8
9  global max_iter m_iter delta c1 c2 N p0 p1 y0 yn cf;
10
11 % tolerances for QN and Wolfe conditions
12 max_iter = 9;
13 m_iter = floor(max_iter/2);
14 delta = 10^(-max_iter);
15 c1 = 10^(-4);    % used in Armijo condition
16 c2 = 0.9;        % used in crvature condition
17 cf = 0.2 % friction coefficient, 0 for none
18 assert(0 < c1 && c1 < c2 && c2 < 1)
19 assert(0 <= cf)
20
21 N = 24;        % number of terms in integral approximate sum
22 h = normpdf(0,2); % test photogenic height
23 p0 = [0; 1];  % starting point
24 p1 = [1; h]   % end point
25 y0 = 1-p0(2); % reflect displacements to account for y-axis direction
26 yn = 1 - p1(2);
27 assert(p0(2) > p1(2)); % so that the particle can slide down
28
29 plot_solutions()
30 % solve and plot
31 function plot_solutions()
32     global N m_iter p0 p1 y0 yn;
33     % solve in reference prame at p0 and pointing toward p1,
34     % so that the analytic solution is given by a standard cycloid.
35     hold on;
36     ax = [0 p1(1) min(0,p1(2)) p0(2)];
37     axis(ax);
38     yline(p1(2), '--');
39     cyc = gen_cycloid();
40     plot(cyc(1,:), cyc(2,:), '-.');
41     % optimize everything except endpoints, which remain fixed
42     y = [1/N: (yn-1/N)/(N-2): yn]'; % intermediate entries.
43     plot(y0:1/N:1, [p0(2);1-y;p1(2)],'.-', 'Color', [1,0,1]);
44 %     y = flipud(sin(acos(y))); % intermediate entries.
45 %     y(1) = y(1)+m_iter/N;
46     plot(y0:1/N:1, [p0(2);1-y;p1(2)],'.-', 'Color', [0,1,1]);
47
48     drawnow;
49     x = 1 - qNewton(y, @time, @dtime);
```

```
50      r = size(x,2);
51      eval(time(cyc(2,:)))
52      % Plot/Animate
53      for k = 2:r
54          pause(2/r);
55  %         t = time(x(:,k));
56          plot(0:1/N:1, [p0(2);x(:,k);p1(2)],'-', 'Color', [1-k/r,k/r,0]);
57          axis(ax);
58      end
59      plot(0:1/N:1, [p0(2);x(:,r);p1(2)],'-', 'Color', [0,0,1]);
60      plot(cyc(1,:),cyc(2,:),'ro', 'Color', 'blue');
61  end
62
63  function cyc = gen_cycloid()
64      global N p1;
65      th0 = 0;    % cycloid's corresponding circle has rotated 0 degrees
            at start
66      syms th1 R; % find cycloid parameters by solving for its symbolic
            vars
67      [th1, R] = solve([p1(1)==R*(th1-sin(th1)), p1(2)==1-R*(1-cos(th1))],
            [th1 R]);
68      th = [th0:th1/N:th1];   % parameter space of angles
69      cyc = [R*(th-sin(th)); 1-R*(1-cos(th))]; % cycloid curve
70  end
71
72  function x = qNewton(xk, g, dg)
73      global max_iter m_iter delta c1 c2 N p1 y0 yn;
74      k = 1;                  % current iteration
75      x(:,k) = xk;            % current estimate
76      d = dg(x(:,k));         % current gradient
77      assert(size(d,2)==1);   % is a vector
78      Bk = eye(size(d,1));        % current Hessian approximation
79      e = 1;                  % "current" error
80      while e>delta && k < 10^max_iter
81          z =  g(x(:,k));     % current value
82          p(:,k) = -Bk\d;     % Newton direction
83
84          while not(is_descent_direction(p(:,k),d))
85              p(:,k) = gradient_direction(d, 1);
86          end
87          ak = 10^(-m_iter); % steplength alpha
88          akc = 0;
89          ac = g(x(:,k)+ak*p(:,k)) <= z+c1*ak*p(:,k)'*d;
90          while not(ac && -p(:,k)'*dg(x(:,k)+ak*p(:,k)) <= -c2*p(:,k)'*d
                || isinf(ak))
91              ak = ak*(1+1/N);
92              akc= akc+1;
93              if akc== 10^(m_iter)
94                  return % because new estimates within margin of error.
95              end
96          end
```

```
97          if isnan(g(x(:,k)+ak*p(:,k)))
98              return % without adding Nan entry
99          end
100
101         s(:,k) = ak*p(:,k);
102         x(:,k+1)=x(:,k)+s(:,k);
103         y(:,k) = dg(x(:,k+1))-d;
104         Bk = BFGS_update(Bk, y(:,k), s(:,k));
105         k = k + 1;
106         z =  g(x(:,k));
107         d = dg(x(:,k));
108         e = norm(d)/(1+norm(d));
109     end
110 end
111
112 function Bkp = BFGS_update(Bk, yk, sk)
113     Bkp = Bk + (yk*yk')/(yk'*sk) - (Bk*(sk*sk')*Bk)/(sk'*Bk*sk);
114 end
115
116 function gd = gradient_direction(d, alpha)
117     gd = -alpha*d;
118 end
119
120 function b = is_descent_direction(p, dfxk)
121     b = p.'*dfxk < 0;
122 end
123
124 function d = dtime(y)
125     n = size(y, 1) + 1;
126     d = zeros(n-1, 1);
127     for i = 1:n-1
128         dy = time([y(1:i-1);y(i)*(1+0.5/n);y(i+1:n-1)]) -  time([y(1:i
                -1);y(i)*(1+-0.5/n);y(i+1:n-1)]);
129         d(i) = dy/(1/n);
130     end
131 end
132
133 function t01 = time(y) % riemann sum of f0 from y0 to yn
134     global y0 yn;
135     n = size(y, 1) + 1;
136     t01 = f0(n,0,y0,y(1)) + f0(n,n,y(n-1),yn);
137     for i = 2 : n-1 % outer terms + inner terms
138         t01 = t01 + f0(n,i, y(i-1),y(i));
139     end
140 end
141
142 function fxk = f0(n,k, yk,ykp) % time elapsed during infinetismal travel
143     fxk = ds(1/n, ykp - yk) / midpoint(n,k,yk, ykp, @vfric); % time =
            distance/(distance/time)
144 end
145
```

```
146 function dist = ds(dx, dy) % Euclidean line element
147     dist = sqrt(dx^2+dy^2);
148 end
149
150 function mid = midpoint(n,k,yk, ykp, v) % midpoint/average velocity from
        time k to kp,
151     mid = (v(n,k,yk) + v(n,k,ykp))/2;
152 end
153
154 function vel = vfric(n,k,yk) % no friction
155     global p0 p1 cf;
156     grav = 9.80665; % constant of gravitational acceleration in m/s^2
157     mu = 0.01;
158     xk = 0;
159     if (cf ~= 0)
160         xk = k*(p1(1)-p0(1))/n;
161     end
162     vel = sqrt(2*grav*(yk-cf*xk));
163 end
```

References

[1] G. Galilei, *Dialogues Concerning Two New Sciences*, ser. Classics of science. Dover, 1954. [Online]. Available: https://books.google.com/books?id=BvyG03NrbC8C

[2] *Acta eruditorum*, ser. Acta eruditorum. Christoph Günther, 1696. [Online]. Available: https://books.google.com/books?id=4q1RAAAAcAAJ

[3] "Philosophical transactions of the royal society of london," vol. 19, p. 424, 1697.

[4] I. Griva, S. Nash, and A. Sofer, *Linear and Nonlinear Optimization: Second Edition*, ser. Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics (SIAM, 3600 Market Street, Floor 6, Philadelphia, PA 19104), 2009. [Online]. Available: https://books.google.com/books?id=uOJ-Vg1BnKgC